

Lab 7: REST APIs, Local Language Models, Agentic Monitoring, and Adaptive Escalation

Lab Learning Outcomes

- Design and implement a REST API using FastAPI.
- Interact with a locally hosted language model through a REST interface.
- Build a simple agentic monitoring loop (sense-reason-act).
- Implement adaptive escalation using deterministic rules combined with model-based reasoning.

Conceptual introduction

REST API

A REST API (Representational State Transfer Application Programming Interface) is an architectural style for building distributed systems over HTTP. It structures communication around resources rather than actions. A resource represents a conceptual entity in the system, such as an order, a user, or a device. Each resource is identified by a URI and manipulated through standard HTTP methods.

Resource-oriented URLs - A well-designed REST API models nouns, not verbs. Instead of writing:

```
/createOrder  
/getOrderById
```

one exposes:

```
/v1/orders  
/v1/orders/{id}
```

Versioning (for example, /v1/) isolates breaking changes and supports long-term maintainability. Resource hierarchies reflect domain structure:

```
/v1/users/{user_id}/orders
```

Such design improves readability, predictability, and scalability.

Stateless requests

Statelessness means each HTTP request contains all necessary information for processing. The server does not store session state between calls. Authentication tokens, request parameters, and identifiers must be included in every request.

Statelessness enables:

- Horizontal scaling (any instance can handle any request)
- Fault tolerance
- Simpler caching strategies
- Reduced coupling between client and server

State is stored in databases or external systems, not in server memory.

Standard HTTP verbs

REST leverages existing HTTP semantics:

GET retrieves a representation of a resource and must be safe and idempotent.

POST creates a new resource or triggers non-idempotent processing.

PUT replaces a resource and is idempotent.

PATCH partially updates a resource.

DELETE removes a resource and is idempotent.

Idempotency means repeating the same request produces the same outcome. That property is essential in distributed systems where retries occur due to network failures.

Meaningful status codes

HTTP status codes communicate outcome clearly:

200 OK – successful retrieval or update

201 Created – new resource created

204 No Content – successful action with no body

400 Bad Request – client-side validation error

401 Unauthorized – authentication required

403 Forbidden – authenticated but not allowed

404 Not Found – resource does not exist

409 Conflict – state conflict

500 Internal Server Error – unexpected server failure

Precise status codes allow clients to implement correct retry logic and error handling.

Separation between client and server

REST enforces separation of concerns:

- Client handles presentation and user interaction.
- Server handles data persistence, validation, and business logic.

Such decoupling allows independent evolution. Front-end applications (web, mobile, IoT) can change without modifying backend services, provided the contract remains stable.

Representation and content negotiation

Resources are transmitted as representations, typically JSON. The same resource can be represented differently depending on context. HTTP headers such as Content-Type and Accept define how data is encoded.

Example:

Content-Type: application/json

Clear representation standards reduce ambiguity and simplify interoperability.

Caching

REST supports HTTP caching mechanisms:

- Cache-Control headers
- ETag and conditional requests
- 304 Not Modified responses

Effective caching reduces latency and server load.

Hypermedia (HATEOAS)

In strict REST theory, responses include links to related actions. While not always implemented in practice, hypermedia enables discoverability and loose coupling.

Example:

```
{
  "id": 10,
  "item": "book",
  "links": {
    "self": "/v1/orders/10",
    "update": "/v1/orders/10",
    "delete": "/v1/orders/10"
  }
}
```

REST in microservice architectures

Modern distributed systems rely on REST APIs to connect independent services. Each microservice exposes a well-defined contract. Services communicate via HTTP, often behind API gateways.

Advantages include:

- Independent deployment
- Language-agnostic interoperability
- Clear domain boundaries
- Easier monitoring and auditing

Observability endpoints

Production systems require observability. Two common endpoints are:

/health - indicates service readiness and liveness

/metrics - exposes performance and error statistics

Health endpoints support orchestration platforms (e.g., container schedulers) in determining whether a service should receive traffic. Metrics endpoints feed monitoring systems and enable automated alerting.

In intelligent monitoring contexts, such as the agentic systems explored in this lab, observability endpoints become inputs to reasoning agents. Rather than simple threshold alerts, metrics can be interpreted semantically by models to guide escalation decisions.

Summary

REST is not merely a transport mechanism. It is a disciplined architectural approach that:

- Encourages scalability through statelessness
- Improves reliability via idempotent operations
- Enhances clarity through resource modelling
- Supports automation through consistent semantics
- Enables monitoring and resilience in distributed systems

Such properties make REST foundational in modern cloud-native and AI-integrated infrastructures.

Language models

A language model estimates the probability distribution of the next token given the preceding context.

Formally, it models:

$$P(\text{token}_n \mid \text{token}_1 \dots \text{token}_{n-1})$$

Modern large language models (LLMs) are typically transformer-based neural networks trained on large-scale corpora. Training is commonly self-supervised: the model learns structure, grammar, semantics, and patterns by predicting masked or next tokens.

From next-token prediction, broader capability emerges. When prompted appropriately, the same probabilistic mechanism supports higher-level tasks.

Core capabilities of modern LLMs

Text classification

A prompt can constrain the model to assign categories, for example:

- Severity levels (low, medium, high)
- Sentiment classes
- Intent detection
- Topic labelling

Example use in monitoring: "Classify the following log message into severity levels."

Summarisation

Models can compress long sequences into shorter representations while preserving meaning. In monitoring contexts, summarisation is useful for:

- Aggregating bursts of logs
- Reducing cognitive load during incidents
- Producing short incident summaries and handover notes

Structured information extraction

LLMs can extract entities and output structured data (often JSON) when constrained. For example:

```
Input: "Order 103 failed due to timeout after 15 seconds."
Output: {
  "order_id": 103,
  "error_type": "timeout",
  "duration_seconds": 15
}
```

Structured extraction is valuable because it allows integration with deterministic code and downstream systems.

Reasoning steps

When prompted to show intermediate steps (e.g., "think step-by-step"), models may produce a chain of reasoning that can improve reliability for tasks such as triage and decision support. A reasoning trace should be treated as a generated artefact rather than a guarantee of correctness.

Limitations

Language models are probabilistic rather than deterministic. Common issues include:

- Hallucinated facts
- Invalid JSON or schema violations
- Sensitivity to prompt wording and formatting
- No built-in mechanism to verify correctness

For safety-critical systems, model outputs must be validated and bounded by deterministic guardrails.

Why use a lightweight local model in this lab

A small model (for example, 1–3 billion parameters) deployed locally via Ollama is sufficient for classification and structured extraction tasks, while remaining fast and practical on modest hardware.

Local deployment benefits

- **Reproducibility:** a stable environment for all students with consistent model behaviour.
- **Data control:** prompts and responses remain inside the local Docker network; no external data exposure.
- **Low-latency inference:** avoids external network round trips, improving responsiveness for monitoring loops.
- **No dependency on external APIs:** no API keys, rate limits, or internet reliance.

Architectural role in this lab

Within the monitoring loop:

- Metrics and system state provide structured input.
- The LLM performs semantic interpretation (triage).
- Deterministic rules enforce safety constraints.
- Escalation logic combines model outputs with hard thresholds and confidence.

The model is not used as an autonomous controller. The model functions as a reasoning component embedded inside a wider control system. Deterministic logic should override unsafe or ambiguous model outputs.

Operational considerations

- Constrain outputs with explicit schemas.
- Validate JSON before acting.
- Separate reasoning from actuation.
- Log prompts and responses for auditability.
- Use confidence thresholds and persistence windows to reduce alert flapping.

Conceptual framing

A language model is a statistical pattern learner. An agent uses a language model as a reasoning module. A robust system surrounds that reasoning module with deterministic controls.

Agentic monitoring

Traditional monitoring systems primarily *measure* and *alert*. They collect telemetry (metrics, logs, traces), evaluate it against static rules (for example, thresholds), and then notify an operator when a rule is violated. Such systems are effective for well-defined failure modes, but they can struggle with ambiguity, noisy signals, and situations where context matters.

Agentic monitoring extends monitoring from passive reporting into *conditional decision-making*. An “agent” does not merely raise an alert; it attempts to interpret what is happening and choose an appropriate next step. The key idea is a simple loop:

Sense collect signals → Reason interpret signals → Act take an action

1) Sense: collect signals (metrics, logs, traces, events)

Sensing concerns telemetry acquisition. Typical inputs include:

- **Metrics:** request rate, error rate, latency percentiles, CPU/memory usage, queue depth.
- **Logs:** structured or unstructured messages, stack traces, error codes, context identifiers.
- **Traces:** distributed tracing spans showing request paths and bottlenecks across services.
- **Events:** deployment notifications, configuration changes, autoscaling actions, dependency outages.

In practice, good sensing also involves basic data hygiene:

- **Sampling:** controlling volume while preserving relevant signal.
- **Correlation IDs:** linking logs/traces to specific requests or user sessions.
- **Time-windowing:** aggregating signals over consistent intervals to reduce noise.
- **Normalisation:** ensuring comparable units and formats (e.g., seconds vs milliseconds).

2) Reason: interpret signals using logic or ML

Reasoning converts raw telemetry into an assessment of system state. Traditional monitoring uses fixed rules: “alert if error rate > 5%.” Agentic monitoring adds more flexible interpretation, for example:

- **Contextual severity:** a 2% error rate may be critical for a payments service but tolerable for a batch job.
- **Pattern detection:** repeated timeouts after a deployment suggests regression rather than random noise.
- **Root-cause hypotheses:** increased latency with stable CPU may indicate downstream dependency issues.
- **Uncertainty estimation:** recognising when evidence is ambiguous and additional analysis is needed.

Reasoning can be implemented with:

- **Deterministic logic:** thresholds, rate-of-change checks, correlation rules, state machines.
- **Statistical methods:** baselines, anomaly detection, change-point detection.
- **ML components:** classifiers for log severity, clustering for incident grouping, language models for triage summaries.

Agentic monitoring should not rely solely on a model. Deterministic “guardrails” typically remain responsible for hard safety constraints, while ML/LLM reasoning provides contextual interpretation and prioritisation.

3) Act: trigger remediation or escalation

Action closes the loop. In a conventional system, the “action” is often only a notification. An agent can take a broader range of actions, including:

- **Enrichment:** fetch extra context (recent deployments, dependency health, top error messages).
- **Deduplication:** group similar alerts into a single incident ticket.

- **Escalation:** page a human operator when severity warrants it, possibly with a concise incident summary.
- **Runbook guidance:** recommend steps ("check database connections", "rollback last deployment").
- **Limited automation:** restart a crashed service, scale out replicas, or temporarily disable a failing feature flag.

Action selection should be risk-aware. Low-risk actions (such as logging, opening a ticket, or fetching diagnostics) are generally acceptable. High-risk actions (such as automatic rollback or configuration changes) require strict safeguards, approval gates, or restricted deployment environments.

Traditional monitoring vs agentic monitoring

Traditional monitoring

- Collect telemetry
- Apply fixed rules (thresholds)
- Send alerts
- Humans interpret and respond

Agentic monitoring

- Collect telemetry
- Interpret context (logic + ML)
- Choose actions based on severity and uncertainty
- Escalate or remediate with guardrails

Why agentic monitoring matters

Modern systems generate large volumes of telemetry and alerts. Without interpretation, operators face alert fatigue. Agentic monitoring aims to reduce noise by prioritising what matters, providing context, and selecting an appropriate operational response.

Summary

Agentic monitoring introduces a sense-reason-act loop. The system becomes capable of conditional decision-making rather than passive reporting. In this lab, the agent reasons over simple service signals and uses an LLM to generate structured triage outputs, while deterministic rules provide safety constraints for escalation.

Adaptive escalation

Escalation determines when and how an incident moves from automated handling to human intervention. Traditional systems often rely on fixed thresholds (for example, "alert if error rate > 5%"). While simple and transparent, static rules can be brittle, generating excessive alerts or failing to capture contextual risk.

Adaptive escalation augments fixed thresholds with contextual reasoning and uncertainty awareness. A robust design typically combines:

- Deterministic guardrails (hard thresholds)
- Model-based reasoning (contextual interpretation)

- Confidence-aware decisions
- Multi-level escalation (light → heavy → human)

1) Deterministic guardrails (hard thresholds)

Guardrails define non-negotiable boundaries. If a critical condition is met, escalation occurs regardless of model output.

Examples:

- Error rate > 20% for 2 consecutive windows.
- Database unreachable.
- Health endpoint returns failure.
- Latency > 5× baseline for sustained duration.

Guardrails provide safety. Model reasoning should never override a clearly unsafe state. Deterministic logic ensures predictable behaviour under extreme conditions.

2) Model-based reasoning (contextual interpretation)

Context frequently determines severity. A moderate error rate during a deployment window may be expected; the same signal during steady-state operation may indicate failure.

Model-based reasoning (for example, via an LLM) can:

- Interpret patterns across multiple signals.
- Summarise recent logs before escalation.
- Infer likely causes based on historical patterns.
- Adjust severity categorisation using contextual clues.

Rather than replacing thresholds, model reasoning complements them by incorporating domain-specific semantics.

3) Confidence-aware decisions

Model outputs are probabilistic. An escalation mechanism should account for uncertainty.

Example policy:

```
if severity == "high" and confidence < 0.7:  
    escalate to heavier analysis  
if severity == "high" and confidence ≥ 0.7:  
    escalate to human
```

Low confidence suggests ambiguity. In such cases, the system may:

- Request additional data.
- Switch to a larger or more specialised model.

- Delay escalation pending further evidence.

Confidence-aware escalation reduces unnecessary human paging while maintaining safety.

4) Multi-level escalation

Escalation should not be binary. A layered structure supports graduated response:

Level 0 — Light

- Log event
- Record metrics
- No human involvement

Level 1 — Heavy (automated enrichment)

- Run deeper analysis
- Query additional diagnostics
- Generate structured incident summary

Level 2 — Human escalation

- Notify on-call engineer
- Provide summarised context
- Attach recommended next steps

Such staging reduces alert fatigue and reserves human attention for genuinely critical cases.

Additional adaptive strategies

Persistence rules

Escalate only if a condition persists across multiple monitoring windows. Short-lived anomalies may self-resolve.

Rate-of-change detection

Sudden spikes may be more critical than steady elevated values.

Environment awareness

Deployment windows, maintenance periods, and known experiments can modify escalation behaviour.

Comparison: static vs adaptive escalation

Static escalation

- Single threshold trigger
- No contextual reasoning
- High false-positive rate
- Binary alert/no-alert logic

Adaptive escalation

- Threshold + contextual reasoning
- Confidence-aware decision-making
- Graduated response levels
- Reduced alert fatigue

Summary

Adaptive escalation integrates deterministic safety constraints with model-based reasoning and uncertainty management. A multi-level structure ensures proportionate response, balancing automation efficiency with operational safety.

Architecture overview

Two containers on the same Docker network:

- NEST container (Jupyter + Python execution)
- Ollama container (local model server)

The Python code runs inside the NEST container. Ollama runs independently but is reachable via internal Docker DNS as: `http://ollama:11434`

Run the docker container

Visit [Docker webpage](#) to learn more about docker.

If you are using your personal laptop - ensure that the steps described in [Install Docker Desktop on Windows machines](#) have been completed successfully.

1. Start docker desktop - **if you are using a lab PC**, ensure that your **DOCKER DESKTOP** is configured to use at least **4GB of RAM (Docker Desktop->Settings->Resources)**! **Ask your lab tutor for support if required!**
2. Create docker-compose.yml:

docker-compose.yml

```
services:
  nest:
    image: pedrombmachado/nest:comp40771
    container_name: comp40771_container
    environment:
      - NEST_CONTAINER_MODE=jupyterlab
      - OLLAMA_BASE=http://ollama:11434
    ports:
      - "8081:8080"
      - "8000:8000"
    volumes:
      - docker_comp40771:/opt/data
    security_opt:
      - seccomp=unconfined
    cap_add:
      - SYS_PTRACE
    depends_on:
      - ollama
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
volumes:
  docker_comp40771:
  ollama_data:
```

3. Start the containers

PowerShell/Mac/Linux terminal

```
docker compose up -d
```

4. Pull lightweight model:

PowerShell/Mac/Linux terminal

```
docker exec -it ollama ollama pull llama3.2:1b
```

```
docker exec -it ollama ollama pull phi3:mini
```

Fork and clone the repository and run the Jupyter notebook

Forking and cloning the project from the git repository

Step 1: Login into <https://olympus.ntu.ac.uk/> using your NTU credentials.

Step 2: Goto the https://olympus.ntu.ac.uk/comp40771/comp40771_lab7 and fork it

Step 3: Goto your account (e.g. N123456/comp40771_lab7) hit the button **Code** and copy the repository url.

Step 4: Open a tab in your favourite browser (e.g. Chrome, Firefox, Edge, Safari, etc.), enter **http://localhost:8081/lab** and open a terminal.

Step 5: Clone the repository

Jupyter lab terminal

```
git clone https://olympus.ntu.ac.uk/YOUR-STUDENT-NUMBER/comp40771_lab7
```

where YOUR-STUDENT-NUMBER is your student number. Enter your student ID in the field Username and your NTU password in the field Password.

Run and analyse the code

Run, study and complete the task in **comp40771_lab7.ipynb**.

Commit and push your source code

Open a terminal and navigate to the folder comp40731_lab7

Jupyter lab terminal

```
cd /opt/data/comp40771_lab7
```

Add all the changes, commit the changes

Jupyter lab terminal

```
git config --global user.name "Student Name"  
git config --global user.email "student@ntu.ac.uk"  
git add *  
git commit -m "update"
```

Push the changes to Olympus

Jupyter lab terminal

```
git push
```

Stop the containers

Let's stop the containers:

PowerShell/Mac/Linux terminal

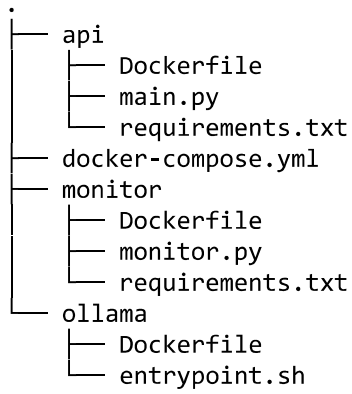
```
docker compose down
```

Apply your knowledge

In this task, you will Build an “agentic” monitoring loop that makes decisions (classify → act) rather than only reporting and implement adaptive escalation using severity + confidence thresholds and fallbacks.

Prepare the environment

On your host machine, create the folder lab 7. The folder structure stucture should be as follows:



Step 1: Create the folder structure

PowerShell/Mac/Linux terminal

```
mkdir lab7
```

```
cd lab7
```

```
mkdir api
```

```
mkdir monitor
```

```
mkdir ollama
```

Step 2: Create docker-compose.yml:

In the folder lab7, create docker-compose.yml using your favourite file editor.

docker-compose.yml

```

services:
  nest:
    image: pedrombmachado/nest:comp40771
    container_name: comp40771_container
    ports:
      - "8081:8080"
    volumes:
      - docker_comp40771:/opt/data
    environment:
      - NEST_CONTAINER_MODE=jupyterlab
      - OLLAMA_BASE=http://ollama:11434
      - API_BASE=http://api:8000
    security_opt:
      - seccomp=unconfined
    cap_add:
      - SYS_PTRACE
    depends_on:
      - ollama
      - api

  ollama:
    build: ./ollama
    container_name: ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama:/root/.ollama
    environment:
      # Optional: override without rebuilding
      - OLLAMA_MODELS=llama3.2:1b phi3:mini

  api:
    build: ./api
    container_name: api
    ports:
      - "8000:8000"
    environment:
      - LOG_LEVEL=INFO
    depends_on:
      - ollama

  monitor:
    build: ./monitor
    container_name: monitor
    environment:
      - API_BASE=http://api:8000
      - OLLAMA_BASE=http://ollama:11434
      - MODEL_LIGHT=llama3.2:1b
      - MODEL_HEAVY=phi3:mini
      - ESCALATION_WEBHOOK=
    depends_on:
      - api
      - ollama

volumes:
  docker_comp40771:
  ollama:

```

Notes

- MODEL_LIGHT should remain small for responsiveness. llama3.2:1b is an example lightweight choice when available in your Ollama registry.
- MODEL_HEAVY can remain optional; escalation can switch to a second local model if pulled, otherwise escalate to "human".

Step 3: Create `api/requirements.txt`:

In the folder `api`, create the `requirements.txt` using your favourite text editor:

requirements.txt

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY monitor.py .

CMD ["python", "monitor.py"]
```

Step 4: Create `api/Dockerfile`:

In the folder `api`, create the `Dockerfile` using your favourite text editor:

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 5: Create `api/main.py`:

In the folder `api`, create the `main.py` using your favourite text editor:

main.py

```

import os
import time
import random
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from prometheus_client import Counter, Histogram, generate_latest, CONTENT_TYPE_LATEST
from starlette.responses import Response

app = FastAPI(title="Demo REST API", version="1.0.0")

REQ_COUNT = Counter("requests_total", "Total requests", ["endpoint", "status"])
REQ_LAT = Histogram("request_latency_seconds", "Latency", ["endpoint"])
ERR_COUNT = Counter("errors_total", "Total errors", ["endpoint", "type"])

class OrderIn(BaseModel):
    item: str
    quantity: int

orders = {}
order_id_seq = 1

def maybe_inject_failure(p: float = 0.0):
    if random.random() < p:
        raise RuntimeError("Injected failure for lab demonstration")

@app.get("/health")
def health():
    return {"status": "ok", "time": int(time.time())}

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)

@app.get("/v1/orders/{order_id}")
def get_order(order_id: int):
    endpoint = "/v1/orders/{id}"
    with REQ_LAT.labels(endpoint=endpoint).time():
        try:
            order = orders.get(order_id)
            if order is None:
                REQ_COUNT.labels(endpoint=endpoint, status="404").inc()
                raise HTTPException(status_code=404, detail="Order not found")
            REQ_COUNT.labels(endpoint=endpoint, status="200").inc()
            return {"id": order_id, **order}
        except HTTPException:
            raise
        except Exception as e:
            ERR_COUNT.labels(endpoint=endpoint, type=type(e).__name__).inc()
            REQ_COUNT.labels(endpoint=endpoint, status="500").inc()
            raise HTTPException(status_code=500, detail="Internal error")

@app.post("/v1/orders", status_code=201)
def create_order(order: OrderIn):
    global order_id_seq
    endpoint = "/v1/orders"
    with REQ_LAT.labels(endpoint=endpoint).time():
        try:
            # Failure injection via query-like environment knob
            fail_rate = float(os.getenv("FAIL_RATE", "0.0"))
            maybe_inject_failure(fail_rate)

            if order.quantity <= 0:
                REQ_COUNT.labels(endpoint=endpoint, status="400").inc()
                raise HTTPException(status_code=400, detail="Quantity must be positive")

            order_id = order_id_seq
            order_id_seq += 1
            orders[order_id] = {"item": order.item, "quantity": order.quantity, "created_at":
int(time.time())}
            REQ_COUNT.labels(endpoint=endpoint, status="201").inc()
            return {"id": order_id, **orders[order_id]}
        except HTTPException:
            raise
        except Exception as e:
            ERR_COUNT.labels(endpoint=endpoint, type=type(e).__name__).inc()
            REQ_COUNT.labels(endpoint=endpoint, status="500").inc()
            raise HTTPException(status_code=500, detail="Internal error")

@app.get("/v1/slow")
def slow():
    endpoint = "/v1/slow"

```

```

with REQ_LAT.labels(endpoint=endpoint).time():
    try:
        # Artificial latency for alerting
        time.sleep(random.uniform(0.2, 1.2))
        REQ_COUNT.labels(endpoint=endpoint, status="200").inc()
        return {"ok": True}
    except Exception as e:
        ERR_COUNT.labels(endpoint=endpoint, type=type(e).__name__).inc()
        REQ_COUNT.labels(endpoint=endpoint, status="500").inc()
        raise HTTPException(status_code=500, detail="Internal error")

```

REST topics:

- Resource-oriented URIs (/v1/orders)
- Appropriate status codes (201, 400, 404, 500)
- Idempotence (GET safe; POST creates)
- Versioning strategy (/v1/...)
- Observability endpoints (/health, /metrics)

Step 6: LM -> Create ollama/Dockerfile:

In the folder monitor, create the Dockerfile using your favourite text editor:

Dockerfile

```

FROM ollama/ollama:latest

# Add a small entrypoint that starts the server and pulls required models into /root/.ollama
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

# Optional: make it easy to change models without rebuilding
ENV OLLAMA_MODELS="llama3.2:1b phi3:mini"

ENTRYPOINT ["/entrypoint.sh"]

```

Step 7: LM -> Create ollama/entrypoint.sh:

In the folder monitor, create the entrypoint.sh.txt using your favourite text editor:

entrypoint.sh

```
#!/bin/sh
set -eu

# Start Ollama server in the background
ollama serve &
SERVER_PID="$!"

# Wait until the HTTP API responds (avoid depending on curl/wget)
# Prefer the Ollama CLI as it is available in the image.
i=0
until ollama list >/dev/null 2>&1; do
    i=$((i+1))
    if [ "$i" -ge 60 ]; then
        echo "Ollama server did not become ready in time." >&2
        kill "$SERVER_PID" 2>/dev/null || true
        exit 1
    fi
    sleep 1
done

# Pull required models (idempotent: fast if already present in /root/.ollama volume)
for m in $OLLAMA_MODELS; do
    echo "Ensuring model is available: $m"
    ollama pull "$m"
done

# Keep container running (foreground)
wait "$SERVER_PID"
```

Step 8: Agentic monitoring and adaptive escalation -> Create monitor/requirements.txt:

In the folder monitor, create the requirements.txt using your favourite text editor:

requirements.txt

```
requests==2.32.3
```

Step 9: Create monitor/Dockerfile:

In the folder monitor, create the Dockerfile using your favourite text editor:

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY monitor.py .

CMD ["python", "monitor.py"]
```

Step 10: Create monitor/monitor.py:

In the folder monitor, create the monitor.py using your favourite text editor:

monitor.py

```

import os
import time
import json
import re
import requests

API_BASE = os.getenv("API_BASE", "http://localhost:8000")
OLLAMA_BASE = os.getenv("OLLAMA_BASE", "http://localhost:11434")
MODEL_LIGHT = os.getenv("MODEL_LIGHT", "llama3.2:1b")
MODEL_HEAVY = os.getenv("MODEL_HEAVY", "phi3:mini")
ESCALATION_WEBHOOK = os.getenv("ESCALATION_WEBHOOK", "").strip()

POLL_SECONDS = 10

def fetch_metrics_text() -> str:
    r = requests.get(f"{API_BASE}/metrics", timeout=5)
    r.raise_for_status()
    return r.text

def parse_metric(metrics_text: str, name: str) -> float:
    # Very small parser: sums all samples for a metric without labels
    total = 0.0
    for line in metrics_text.splitlines():
        if line.startswith("#"):
            continue
        if line.startswith(name + "(") or line.startswith(name + " "):
            parts = line.split()
            if len(parts) >= 2:
                try:
                    total += float(parts[-1])
                except ValueError:
                    pass
    return total

def ollama_generate(model: str, prompt: str) -> str:
    payload = {"model": model, "prompt": prompt, "stream": False}
    r = requests.post(f"{OLLAMA_BASE}/api/generate", json=payload, timeout=30)
    r.raise_for_status()
    return r.json().get("response", "").strip()

def triage_with_llm(model: str, context: dict) -> dict:
    prompt = (
        "You are an SRE triage assistant.\n"
        "Given the monitoring context, output STRICT JSON with keys:\n"
        "severity (low|medium|high|critical), confidence (0-1), summary, recommended_action.\n"
        "No extra text.\n\n"
        f"Context:\n{json.dumps(context, indent=2)}\n"
    )
    raw = ollama_generate(model, prompt)
    # Best-effort JSON extraction
    m = re.search(r"\{.*\}", raw, re.DOTALL)
    if not m:
        return {
            "severity": "medium",
            "confidence": 0.3,
            "summary": "Model output not parseable; defaulting to conservative triage.",
            "recommended_action": "Inspect metrics manually; enable heavy escalation if persistent."
        }
    try:
        return json.loads(m.group(0))
    except json.JSONDecodeError:
        return {
            "severity": "medium",
            "confidence": 0.3,
            "summary": "Model JSON decode failed; defaulting to conservative triage.",
            "recommended_action": "Inspect metrics manually; enable heavy escalation if persistent."
        }

def should_escalate(assessment: dict, error_rate_delta: float, latency_hint: bool) -> str:
    sev = assessment.get("severity", "medium")
    conf = float(assessment.get("confidence", 0.0))
    # Deterministic guardrails
    if error_rate_delta >= 5:
        return "human"
    if latency_hint and sev in ("high", "critical"):
        return "human"
    # Confidence-based escalation
    if sev in ("high", "critical") and conf < 0.7:
        return "heavy"
    if sev == "medium" and conf < 0.5:

```

```

    return "heavy"
    return "none"

def notify_human(message: str):
    print(f"[ESCALATE:HUMAN] {message}")
    if ESCALATION_WEBHOOK:
        try:
            requests.post(ESCALATION_WEBHOOK, json={"text": message}, timeout=5)
        except Exception:
            print("[WARN] Webhook failed")

def main():
    print("[monitor] starting")
    prev_errors = None
    prev_reqs = None

    while True:
        try:
            metrics = fetch_metrics_text()
            errors_total = parse_metric(metrics, "errors_total")
            reqs_total = parse_metric(metrics, "requests_total")

            if prev_errors is None:
                prev_errors, prev_reqs = errors_total, reqs_total
                time.sleep(POLL_SECONDS)
                continue

            errors_delta = errors_total - prev_errors
            reqs_delta = max(reqs_total - prev_reqs, 0.0)

            # Simple heuristics for "latency hint" based on histogram buckets not parsed here.
            latency_hint = False

            context = {
                "window_seconds": POLL_SECONDS,
                "requests_delta": reqs_delta,
                "errors_delta": errors_delta,
                "api_base": API_BASE
            }

            assessment_light = triage_with_llm(MODEL_LIGHT, context)
            decision = should_escalate(assessment_light, error_rate_delta=errors_delta,
latency_hint=latency_hint)

            print(f"[monitor] light_assessment={assessment_light} decision={decision}")

            if decision == "heavy":
                assessment_heavy = triage_with_llm(MODEL_HEAVY, context)
                print(f"[monitor] heavy_assessment={assessment_heavy}")
                if assessment_heavy.get("severity") in ("high", "critical"):
                    notify_human(json.dumps(assessment_heavy))
            elif decision == "human":
                notify_human(json.dumps({"light_assessment": assessment_light, "context": context}))

            prev_errors, prev_reqs = errors_total, reqs_total

        except Exception as e:
            print(f"[monitor] error={type(e).__name__} msg={e}")

            time.sleep(POLL_SECONDS)

if __name__ == "__main__":
    main()

```

Agentic behaviour

- Sense: reads metrics on a schedule
- Think: LLM produces a structured triage assessment
- Act: escalation decision triggers a second pass or a human notification
- Guardrails: deterministic thresholds override model outputs

Step 11: Build and run the containers

PowerShell/Mac/Linux terminal

```
docker compose up -d --build
```

Step 12: Verify models are in the volume

PowerShell/Mac/Linux terminal

```
docker exec -it ollama ollama list
```

Advanced tasks

Task A: REST API Enhancements

Container: api

1. Add PUT /v1/orders/{id} - Implement in: api/main.py
 - Add the new route definition.
 - Add validation logic.
 - Update in-memory or database storage logic.
 - Return correct HTTP status codes (200, 400, 404).
2. Add pagination GET /v1/orders?limit=&offset=- Implement in: api/main.py
 - Add new GET endpoint.
 - Parse and validate query parameters.
 - Slice internal order storage using limit and offset.
 - Return metadata: total, limit, offset.
3. Standardise error responses - Implement in: api/main.py
 - Add global exception handlers using @app.exception_handler.
 - Override RequestValidationError handler.
 - Ensure all endpoints return consistent structured error format.

Task B: LLM Integration via REST

Container: api

Primary files:

- Add POST /v1/triage - Implement in: api/main.py
- Add new endpoint definition.
- Use requests to call Ollama at <http://ollama:11434/api/generate>.
- Extract the model's response.

- Return structured JSON only.

2. Enforce response schema - Implement in: api/main.py

- Define a Pydantic model (e.g., TriageResponse).
- Validate model output before returning.
- Return 502 or 500 if schema validation fails.

Ollama container is not modified.

Task C: Agentic Monitoring

Container: monitor

1. Detect high latency -> Modify monitor/monitor.py

- Add timing logic around calls to `http://api:8000/v1/slow`
- Parse latency metrics from `/metrics` endpoint.
- Define threshold values and severity classification logic.

2. Implement runbook actions -> Modify monitor/monitor.py

- Add conditional logic after triage decision.
- Implement:
 - Low → print structured log entry
 - Medium → print simulated ticket creation
 - High/Critical → print simulated page event

No changes required in nest container for this task.

Task D: Adaptive Escalation

Container: monitor

1. Confidence floor rule -> Modify escalation logic in monitor/monitor.py

- Add rule comparing severity and confidence.
- If confidence below threshold, escalate to heavy model or human.

2. Persistence logic -> Modify monitor/monitor.py

- Maintain internal state (e.g., counter for consecutive medium alerts).
- If medium persists for N cycles, escalate to high.
- Reset counter when severity drops.

Task E: Jupyter Notebook Interaction

Container: Jupyter container

API container via `http://api:8000`

Implement the following code in Lab 7 notebook cell

1. Define configuration and helper functions

```
API_BASE = "http://api:8000"
```

- A helper show(title, response) that prints:
 - title
 - HTTP status code
 - parsed JSON body (or raw text if JSON parsing fails)
- Wrapper functions:
 - create_order(item, quantity) → POST /v1/orders
 - get_orders(limit, offset) → GET /v1/orders?limit=&offset=
 - update_order(order_id, quantity) → PUT /v1/orders/{id}
 - triage(prompt) → POST /v1/triage

2. Demonstrate valid REST calls

- GET /health returns 200 and JSON with "status": "ok"
- POST /v1/orders with valid data returns 201 and includes an id
- GET /v1/orders?limit=2&offset=0 returns 200 and includes:
 - items list
 - total, limit, offset
- PUT /v1/orders/{id} with a valid new quantity returns 200 and shows updated quantity
- POST /v1/triage with a prompt returns 200 and a JSON object that matches the triage schema:
 - severity, confidence, summary, recommended_action

3. Demonstrate invalid REST calls (and confirm error codes) -> include calls that intentionally fail and print the returned error JSON:

- POST /v1/orders with quantity = 0 must return 400 (or your chosen validation code) and structured error JSON
- PUT /v1/orders/{id} with quantity = -1 must return 400 and structured error JSON
- GET /v1/orders/999999 (non-existent) must return 404 and structured error JSON
- GET /v1/orders?limit=-1&offset=0 must return 400 and structured error JSON
- POST /v1/triage with an empty or malformed prompt must return 400 and structured error JSON (or 422 if you rely on Pydantic)

Container Responsibility Summary

Jupyter container

- Runs Jupyter notebook
- Acts as client

api container

- Implements REST API
- Integrates with Ollama
- Handles validation and structured errors

ollama container

- Hosts language models
- Provides REST generation endpoint

monitor container

- Implements agentic monitoring
- Applies adaptive escalation logic
- Interacts with API and Ollama

This task contributes to your assessed coursework. Ensure that you save your completed solution in your private CWK repository.